

ROS2 Module

Laboratory Report

TP1: Pick & Place with UR3e (Polyscope)

TP2: Connecting UR3e to a PC and Control with ROS2

Master 2 Robotics and Artificial Perception

Group Members:

- BOUCHELIT, Youcef
- LOUNAS, Gana
- OULD OULHADJ, Reda
- HAOUAM, Yacine
- NORTIER, Jérôme
- OUNALLY, Mohamed Dhia

Date: January 09, 2026

Université Clermont Auvergne

Executive Summary

This laboratory report documents our hands-on experience with the Universal Robots UR3e collaborative robot, covering both basic manipulation through Polyscope programming and advanced ROS2 integration. The work was divided into two complementary sessions:

TP1 focused on mastering Polyscope programming to create an autonomous pick-and-place application to fill a chocolate box. We explored safety protocols, coordinate frame management, TCP calibration, and implemented sophisticated palletizing patterns with contact sensing.

TP2 extended our control capabilities by connecting the UR3e to a PC running ROS2, enabling external control through the UR driver and MoveIt framework. This integration demonstrates the transition from teach-pendant programming to programmatic robotic control.

💡 Learning Outcomes

- Mastered UR3e safety procedures and Polyscope programming environment
- Implemented frame-based programming for flexible task reconfiguration
- Integrated force/contact sensing for robust picking operations
- Successfully established ROS2 communication with industrial hardware
- Demonstrated MoveIt trajectory planning and execution

Contents

Executive Summary	1
1 Context and Objectives	3
2 TP1: Pick and Place in Polyscope	3
2.1 System Startup, Safety and Tool Connection	3
2.2 Manual Manipulation: The 4 Movement Modes	3
2.3 Base vs Tool Frames, Feature Menu and Active TCP	4
2.4 TCP Definition and Payload Estimation	4
2.5 Creating the Work Frames (Features) for the Pile and the Box	4
2.6 Test Program and Motion Types: MoveJ vs MoveL vs MoveP	5
2.7 Building the Pick and Place Program (Palletizing + Suction)	5
2.8 One Full Cycle	6
2.9 Program Structure (What We Had on the Polyscope Tree)	7
2.10 Adding Effort/Contact Sensing (Tool Contact) During Picking	8
2.11 The "Back to PosInit" Issue and How We Fixed It	8
2.12 Final Version and the 3×3 Challenge	9
2.13 Discussion and Answers to the Questions	10
3 TP2: Connecting the UR3e to a PC and Controlling it with ROS2	11
3.1 Network Setup and Basic Communication	11
3.2 External Control URCap and ROS2 Driver	11
3.3 MoveIt / RViz Test	12
4 Conclusion	13
Appendix: Program Running	14

1 Context and Objectives

During these two lab sessions, we worked with a UR3e collaborative robot from Universal Robots and we tried to go from basic use in Polyscope to real control from a PC. To do this, we used ROS2 with MoveIt and we set up the IP address. The global goal was to fill a chocolate box before connecting the robot to a computer and command it through the ROS2 driver (and MoveIt) so we could see the same robot in RViz and actually execute trajectories.

⚙️ System Overview

Hardware:

- Robot: Universal Robots UR3e (6-DOF collaborative robot)
- End-effector: Schmalz suction gripper
- Controller: UR teach pendant running Polyscope

Software Stack:

- Polyscope (UR native programming environment)
- ROS2 (Robot Operating System 2)
- MoveIt (Motion planning framework)
- RViz (3D visualization tool)

2 TP1: Pick and Place in Polyscope

2.1 System Startup, Safety and Tool Connection

First, we powered on the robot and waited for Polyscope to boot. Then, we followed the normal startup sequence to get the UR3e ready (release brakes, initialize and check that we are in a safe state). Safety first.

Once ready, we connected the suction gripper (Schmalz). We went to **Installation** and then **General > Tool I/O** and we selected the gripper controller as the I/O interface. We verified that the gripper status became *connected* before tuning the basic parameters. Afterwards, we tested the suction and release functions in manual mode to ensure the tool responded correctly before teaching points.

⚠️ Important

Safety First: Before touching anything else, we reduced the robot speed to 40% with the speed slider to ensure safety during teaching. This is a critical step when working with collaborative robots in manual mode.

2.2 Manual Manipulation: The 4 Movement Modes

Once set up, we went to the **Move** tab and practiced the four ways of moving the robot:

- **Cartesian manipulation:** we move the TCP in space along the selected frame using the translation/rotation arrows.
- **Joint manipulation:** we move one joint at a time, which is useful for quick reconfiguration or getting out of singularities.
- **Move by coordinates:** we enter (X, Y, Z, RX, RY, RZ) and the robot goes to that pose.

- **Freedrive:** we physically guide the arm by hand (brakes released), which is very practical to roughly position the robot before teaching precise points. (We need to press the button behind the Polyscope to enable freedrive)

2.3 Base vs Tool Frames, Feature Menu and Active TCP

After that we focused on coordinate frames.

We used the **Feature** menu to switch the reference frame used by the manipulation arrows and by coordinate input. When **Feature** = **Base**, the axes are fixed in the robot base so moving in X always means the same real direction in the cell. When **Feature** = **Tool**, the axes follow the end-effector orientation so moving in $Z-$ means "go down along the tool axis".

TCP Importance

The **Active** TCP matters a lot: Polyscope controls the *TCP point* and not the geometry itself. Therefore, if the TCP is wrong, all of our taught points and linear moves are wrong too. That is why we had to calibrate the TCP for the tool and activate the correct one before teaching.

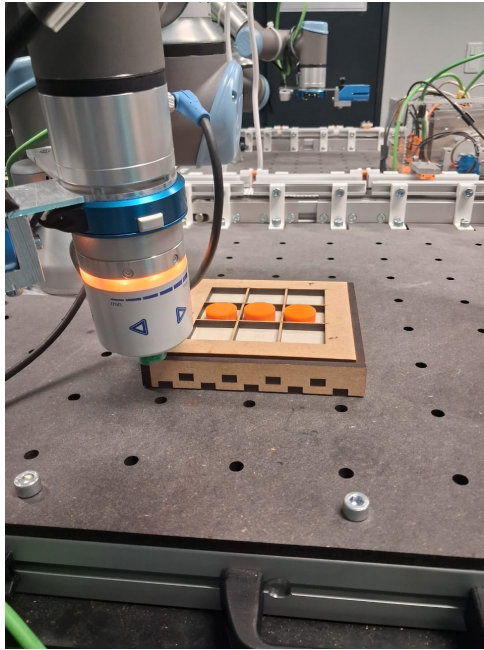
2.4 TCP Definition and Payload Estimation

We created and checked the TCP for the teaching tip (used to define frames) and later we activated the suction TCP (ventouse).

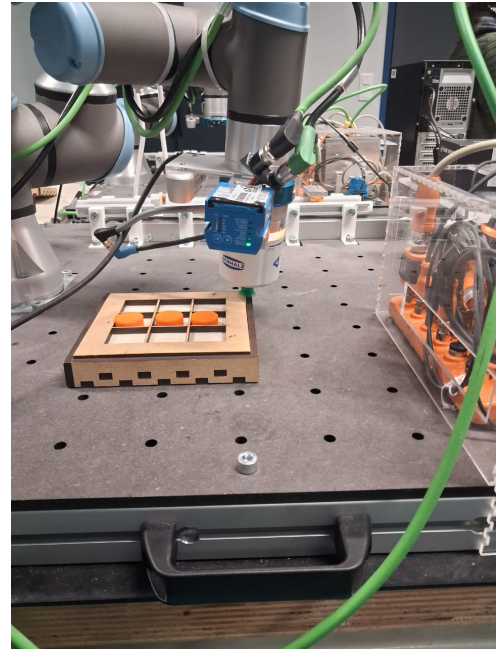
Then, we estimated the payload mass using the wizard. The main idea is that the robot needs a good payload model for gravity compensation, smooth motion and correct force estimation. In practice, we followed the steps in the TP and let Polyscope compute the value before saving it in **Installation > General > Payload**.

2.5 Creating the Work Frames (Features) for the Pile and the Box

We then defined the frames for the main objects: the pile of chocolates and the chocolate box. To do this, we used the 3-point teaching method: first a reference point (origin), then a point on the X direction and finally a third point to define the XY plane. After that, we checked the axis preview to make sure $Z+$ is "up" and X points in the direction we expect. This gave us frames like **Plan_Pile** and **Plan_Boite** (and later **Plan_Boite_2** when we changed the box).



(a) Teaching one corner.



(b) Checking the setup from another view.

Figure 1: Teaching the box feature: we place the TCP on corners to define the frame.

2.6 Test Program and Motion Types: MoveJ vs MoveL vs MoveP

Before doing the full pick and place, we created a small test program to validate the frames and get a feeling of the differences between motion types:

⚙ Motion Type Comparison

- **MoveJ (Joint-space interpolation):** Usually fast and smooth for joints, but the TCP path in space is not necessarily straight. This can create "wide" trajectories and unexpected swings near obstacles.
- **MoveL (Cartesian linear motion):** Linear motion of the TCP between waypoints. It is the good choice for approach/retreat and for keeping a straight line above the box or the pile. It is often slower than MoveJ because constraints are in Cartesian space.
- **MoveP (Process linear motion):** Linear motion with constant speed and blending (circular smoothing) through waypoints. The robot does not fully stop at intermediate points, so the motion is more continuous and the cycle time is better for repetitive trajectories (typical palletizing paths).

When we compared them, we clearly saw that **MoveJ** is convenient to move between far poses, but close to the chocolates we preferred **MoveL** (straight approach) and **MoveP** (smoother continuous moves between approach points). It feels safer and more repeatable.

2.7 Building the Pick and Place Program (Palletizing + Suction)

Now that everything had been analyzed and set up, we turned our attention to the core objective: developing the pick-and-place program. The program was developed step by step, with regular testing of small sections and frequent saving, since a single incorrectly taught point can be very

time-consuming.

First we created a safe initial waypoint `PosInit` above the workspace (high Z , no collision) so the robot could start from a known configuration. We then added a `Palletizing` template for the **pile** in `Depalletizing` mode and we selected `Feature = Plan_Pile`. The next step was to specify the number of items (3 piles in a line) and define `StartItem` and `EndItem`. After that, we placed the approach/exit points so the robot could come from above and leave safely.

Inside `At Each Item`, we inserted the suction action (URCap Schmalz) so the robot actually grabs the chocolate.

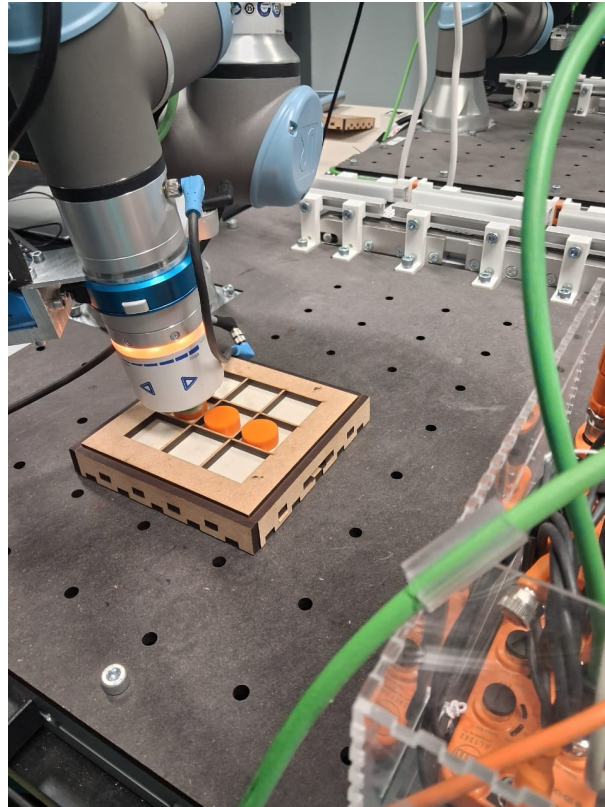


Figure 2: Pick phase: the suction cup reaches the chocolate and vacuum is enabled.

Eventually, we created a second `Palletizing` template for the **box** (in `Palletizing` mode), with `Feature = Plan_Boite`. We taught the first and last cavities and we used the generated movements to approach, release, and retreat.

2.8 One Full Cycle

In summary, one cycle is organized as follows:

- 1) Go above the current pile position (approach point)
- 2) Go down in a straight line until we touch the chocolate (tool contact)
- 3) Start suction and lift up
- 4) Move above the target cavity in the box
- 5) Go down and release
- 6) Go back up

Then, the cycle starts over until the palletizing pattern is completed.

2.9 Program Structure (What We Had on the Polyscope Tree)

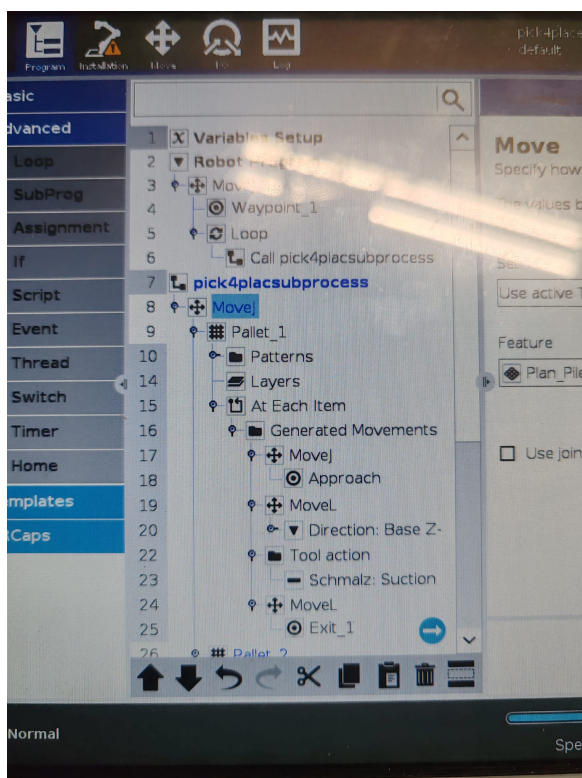
To keep things as clear as possible, we maintained a clean program tree instead of a long list of commands.

Therefore, we started with **Variables Setup** and a **Robot Program** root. Then, we added an initial move (MoveJ to a waypoint) to reach a safe state, while the main operations take place inside a **Loop**. Within that loop, we didn't repeat all the instructions but reused a subprogram (named **pick4placesubprocess**) so the main page remains readable.

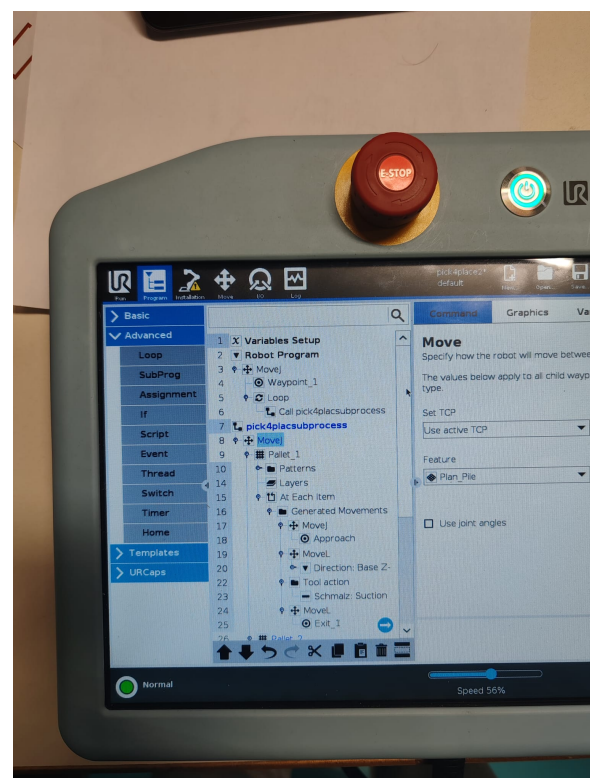
✓ Good Programming Practice

In this subprogram, we can find two palletizing blocks:

- **Pallet_1 (pile / depalletizing):** Feature = Plan_Pile. It contains Patterns, Layers and At Each Item. In this first block, Polyscope generates Approach and Exit moves. We modify those to be mostly MoveL so it moves in a straight line above the stack. For the actual pick we used a Direction move along Base Z- with Until: Tool Contact, immediately followed by activating suction.
- **Pallet_2 (box / palletizing):** Here the idea is the same but with Feature = Plan_Boite (or Plan_Boite_2). We go to Approach before going down to the cavity, then Release and finished by going back up.



(a) Main tree: loop + call to the subprogram.



(b) Subprogram: pallet blocks + generated movements.

Figure 3: Polyscope screenshots of our program structure.

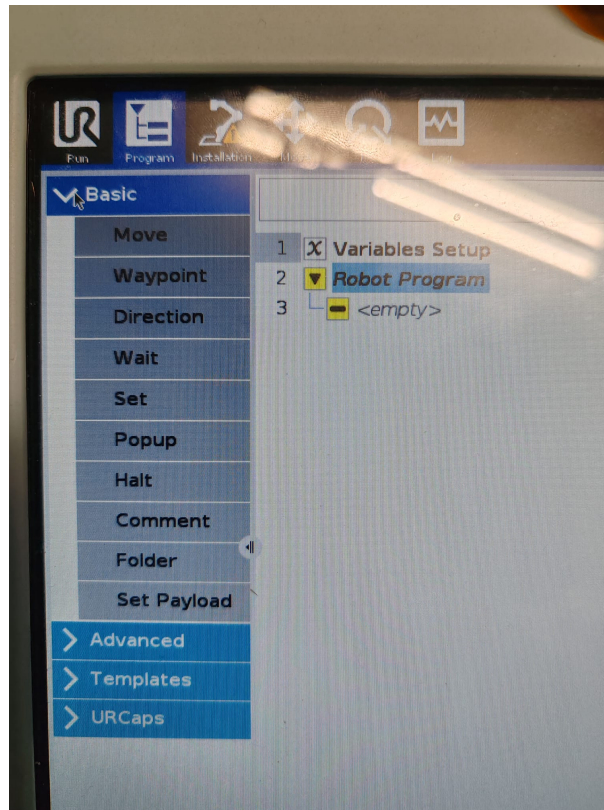


Figure 4: Polyscope programming menu: basic blocks, advanced blocks (loop, subprogram), templates and URCaps.

2.10 Adding Effort/Contact Sensing (Tool Contact) During Picking

For our system to be more robust than just relying on a geometric "tool action point", we implemented an effort-based contact detection during picking.

We replaced the direct "go to contact point" with a **Direction** move along Z — in the **Base** frame, adding a condition to stop using **Until: Tool Contact**. The speed was set to ≈ 99 mm/s to comply with the lab constraint.

2.11 The "Back to PosInit" Issue and How We Fixed It

During the first runs, we noticed a behavior that was wasting time: after placing a chocolate, the robot was going back to **PosInit** before continuing.

What was happening is that our program structure was restarting from the top, executing the **Move** to the initial waypoint in every iteration. In order to get rid of this issue, we had to reorganize the program so that **PosInit** is only executed once at the beginning, before starting the pick-and-place **Loop**. This was effective, as the robot no longer returned to the initial pose between actions.

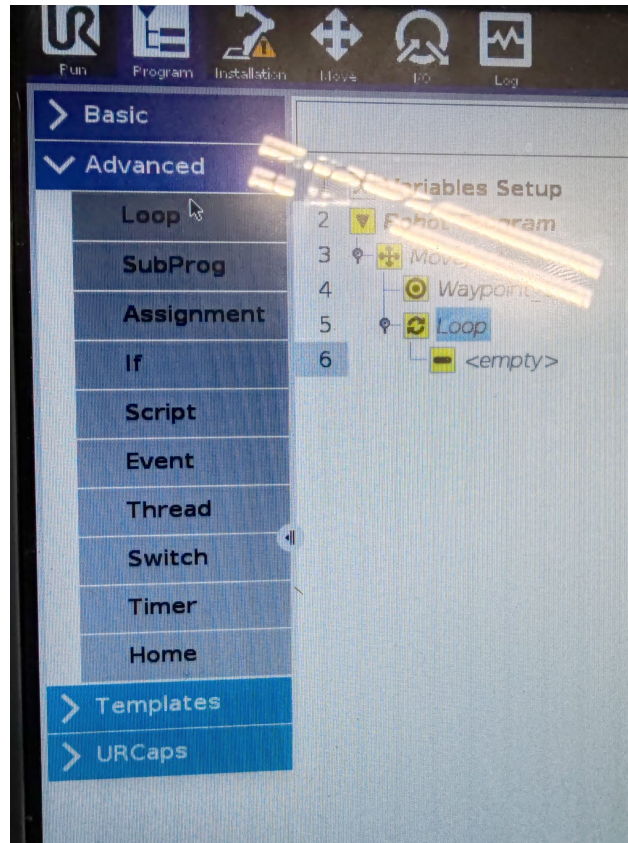


Figure 5: Fixing the behavior by structuring the program with a loop (initial move outside, repeated task inside).

2.12 Final Version and the 3×3 Challenge

Finally, we moved to the full setup: 3 piles with 3 chocolates each and a 3×3 chocolate box. That means:

- For the pile palletizing: 3 piles in a line, **3 layers** (because chocolates are stacked) and safe approach/exit.
- For the box palletizing: a **3×3 pattern** (grid) and a release action in each cavity.

After simply re-teaching the new box feature (Plan_Boite_2), the program executed correctly and successfully filled the box.

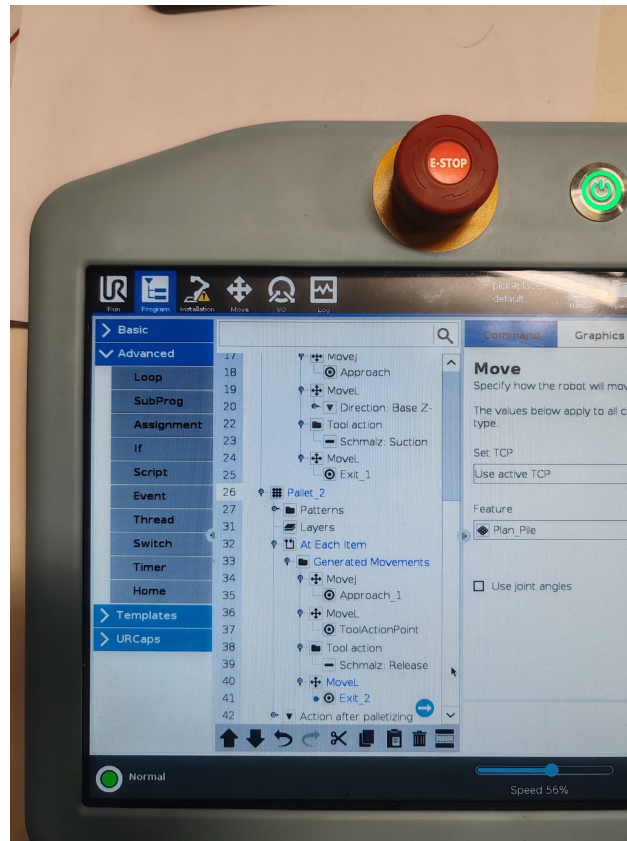


Figure 6: Final program tree: two palletizing blocks (pile + box) with tool actions.

2.13 Discussion and Answers to the Questions

1) Why does the robot return to PosInit before taking a chocolate? How to avoid it?

We saw this because the program structure made the execution restart from the beginning, so the initial move was always repeated. We avoided it by placing `PosInit` outside the loop and looping only the useful sequence (pick from pile then place in the box). Another clean solution in Polyscope is to use `Add action after palletizing` and insert a `Halt` (or a state update) so the robot does not re-run the palletizing from the start when the pattern is finished.

2) How to optimize the robot movements?

To maximize operational efficiency, we optimized the robot's trajectories by restructuring the program to execute the `PosInit` command only during startup—thereby eliminating redundant "home" moves—and by implementing a hybrid motion strategy. This strategy utilizes high-speed **MoveJ** for long-distance transitions, strictly vertical **MoveL** for safe approaches to the source and destination, and **MoveP** with defined blending radii to maintain a constant velocity profile and avoid time-consuming full stops at intermediate waypoints. Furthermore, we minimized the total cycle time by tuning the retreat altitude to the lowest safe clearance height, effectively reducing non-productive vertical travel.

3) Propose a new end-effector to optimize filling the box.

With one suction cup we pick one chocolate at a time. A better approach is to use a two-finger gripper to grab multiple chocolates from the stack (they are aligned and on top of each other), then quickly bring them near the box and finally place each one in its cavity with smaller local motions. Another option is a multi-cup vacuum tool that can pick several chocolates at once, but it needs spacing matching the objects.

4) We moved the box/pile and redefined the frames: does it still work? Why?

Yes, after redefining `Plan_Pile` and `Plan_Boite_2`, it worked again because the program was written relative to these features (frames). So when the objects move, we only update the frame definition and all palletizing waypoints are automatically recomputed in the new pose. If we had taught everything in `Base`, we would have needed to re-teach every point manually.

5) Why must the speed not exceed 100 mm/s for force/contact detection?

The speed limit is a critical constraint dictated by the physics of the robot's internal force estimation. Since `Tool Contact` relies on joint torque monitoring rather than external sensors, higher TCP speeds increase kinetic energy and stopping distances due to inherent filtering and controller latencies. Maintaining a velocity below 100 mm/s ensures that the collision impulse remains within safe thresholds, preventing mechanical damage and ensuring the robot stops immediately upon contact without overshooting.

3 TP2: Connecting the UR3e to a PC and Controlling it with ROS2

3.1 Network Setup and Basic Communication

First we enabled networking on the UR3e from the teach pendant (`Setup Robot > Setup Network`). Then we connected the robot and the PC using an Ethernet cable.

On the PC we configured a manual IPv4 address on the wired interface in order to stay in the same subnet as the robot:

Network Configuration

- Robot IP: 192.168.1.102
- PC IP: 192.168.1.101
- Netmask: 255.255.255.0

Then we verified connectivity with `ping 192.168.1.102`.

3.2 External Control URCap and ROS2 Driver

Then we created a Polyscope program and we inserted the `External Control` program node. This node makes the robot ready to accept control commands from the external ROS2 driver and we configured the host IP and port.

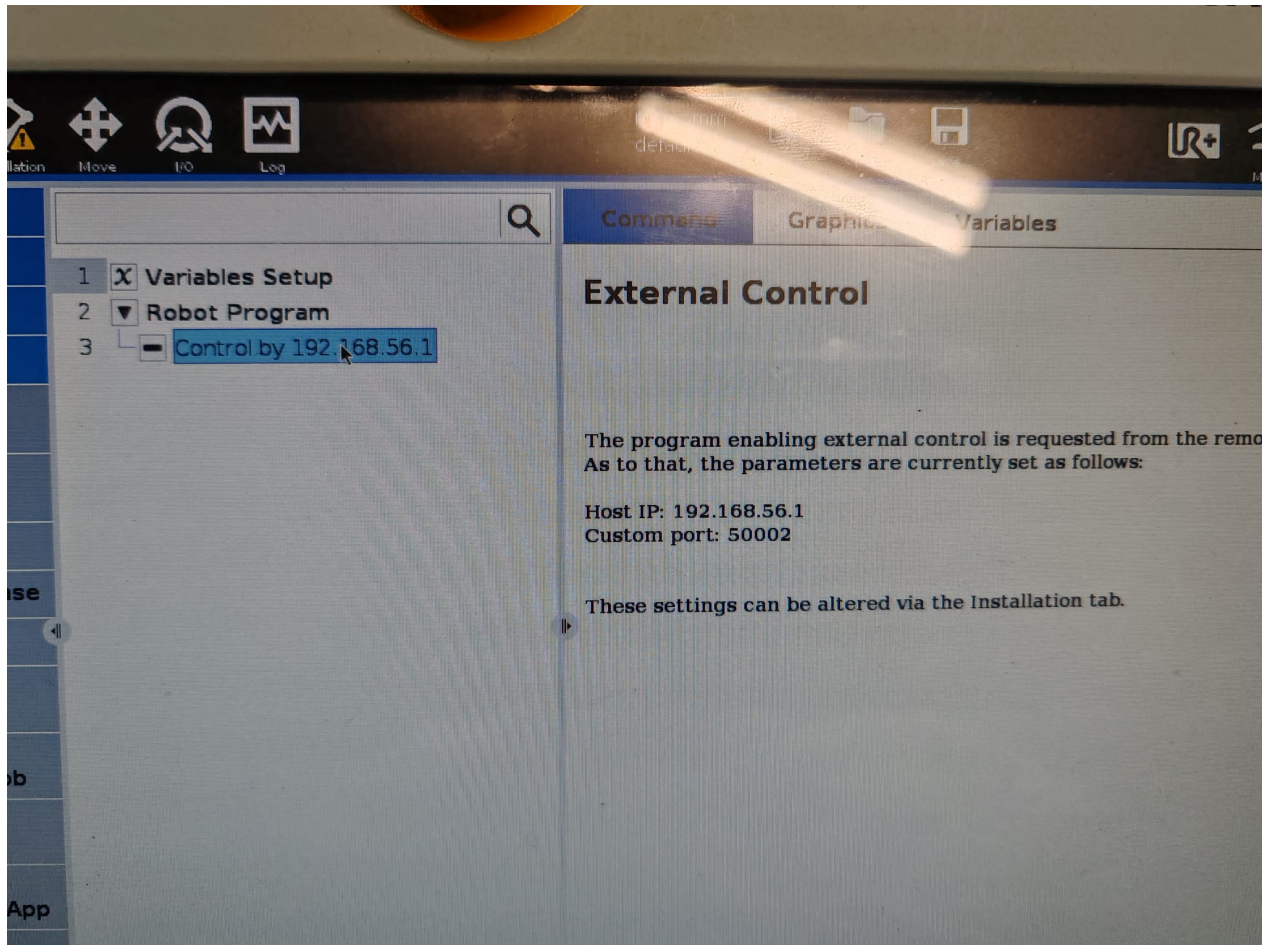


Figure 7: External Control node: the robot is prepared to receive commands from the PC.

On the PC, we launched the ROS2 driver (with the correct robot type and IP), for example:

Listing 1: Launching the UR ROS2 driver

```
1 ros2 launch ur_robot_driver ur_control.launch.py \  
2   ur_type:=ur3e robot_ip:=192.168.1.102
```

After the driver was running and the program on the teach pendant was started, the robot was ready to receive trajectories.

3.3 MoveIt / RViz Test

Then we launched the MoveIt configuration in RViz using:

Listing 2: Launching MoveIt with RViz

```
1 ros2 launch ur_moveit_config ur_moveit.launch.py \  
2   ur_type:=ur3e launch_rviz:=true
```

✓ Validation Steps

We checked that the visualization matches the real robot and we also tested planning and execution:

- Moved joints in RViz
- Planned trajectories using MoveIt
- Executed them so the real robot followed the commanded motion
- Validated that moving the real robot by hand is reflected in RViz, confirming that joint states are properly streamed

4 Conclusion

In TP1 we learned how to set up a cobot (in particular the UR3e), we defined tool parameters (TCP and payload), created work frames and built a full palletizing pick-and-place program with a suction gripper and contact detection. And then in TP2 we connected the UR3e to a PC and we moved from Polyscope programming to ROS2 external control with the UR driver and MoveIt.

🤖 Collaborative Robotics Insight

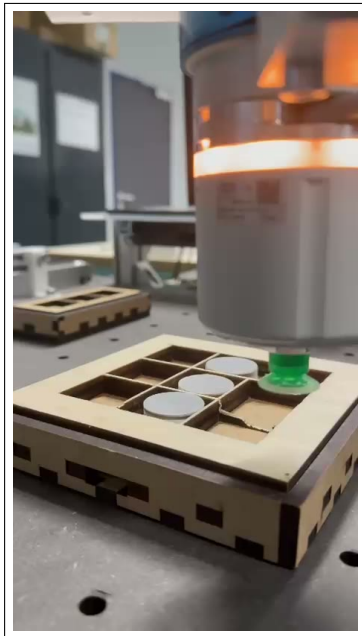
A defining characteristic of collaborative robots (cobots), as opposed to traditional industrial robots, is their strict adherence to safety-rated monitored speed and force-limiting constraints (ISO 15066). These systems integrate sophisticated impedance control and sensitive force-torque monitoring to ensure that kinetic energy remains below injury thresholds. This inherent safety allows for a "fenceless" workspace where humans and robots can interact dynamically without the need for physical barriers, prioritizing human ergonomics and operational flexibility.

Key Takeaways:

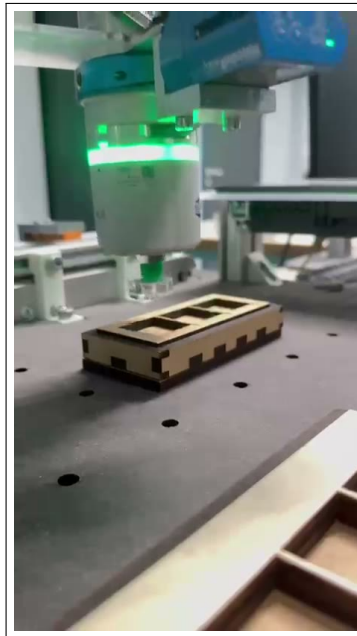
- Frame-based programming enables flexible reconfiguration without re-teaching
- Contact sensing significantly improves robustness in pick operations
- ROS2 integration provides powerful programmatic control capabilities
- Proper program structure is essential for efficient cycle times
- Safety must always be the primary consideration in collaborative robotics

Appendix: Program Running

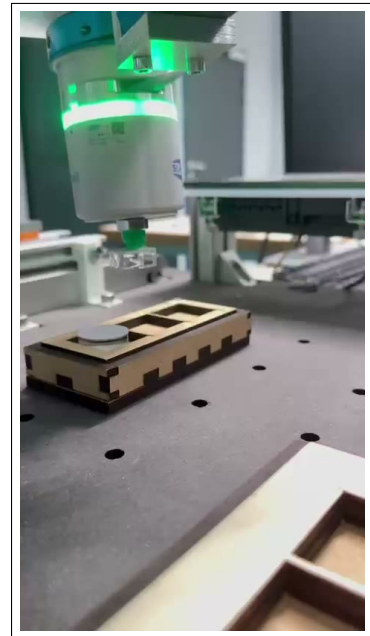
At the end we recorded a short video of the final program which takes chocolates and places them in a box in a loop. We extracted a few frames from it to show the flow (approach, contact, pick, place).



(a) Picking from the box.

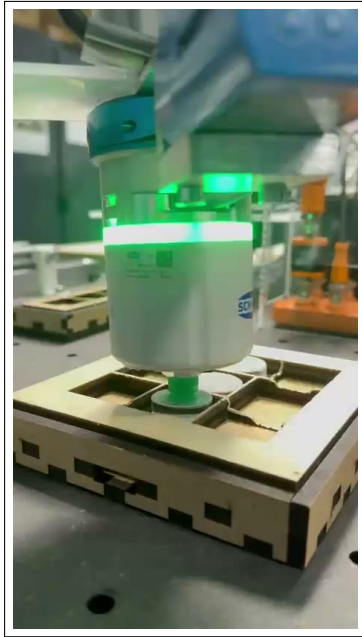


(b) Taking chocolate to the stack.



(c) Placing chocolate in an empty square.

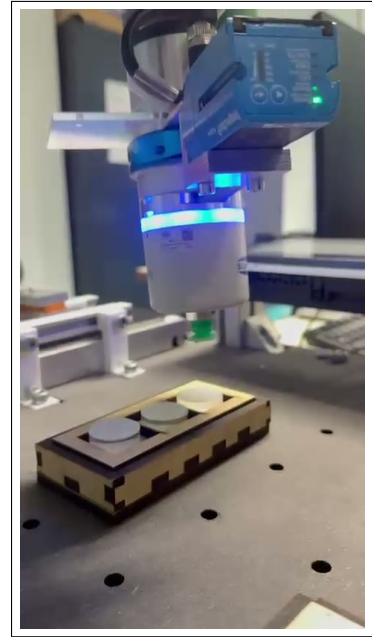
Figure 8: First half of the sequence showing the pick and transfer phases.



(a) Contact with the chocolate and suction.



(b) Travel.



(c) Stacking complete.

Figure 9: Frames from the video demonstration showing the complete sequence.